

Money: structure & flow

How a customer's payment reaches a salon owner's Mobile Money, what holds the money in between, and the specific ways that can go wrong.

September 2026 · for the Bookly team and for Paystack · describes the code as built

01

What changed, and why

Customers now pay into Bookly's Paystack account, not each salon's.

A salon owner never creates a payment gateway account or handles an API key. Text messages moved the same way, onto Bookly's Arkesel account.

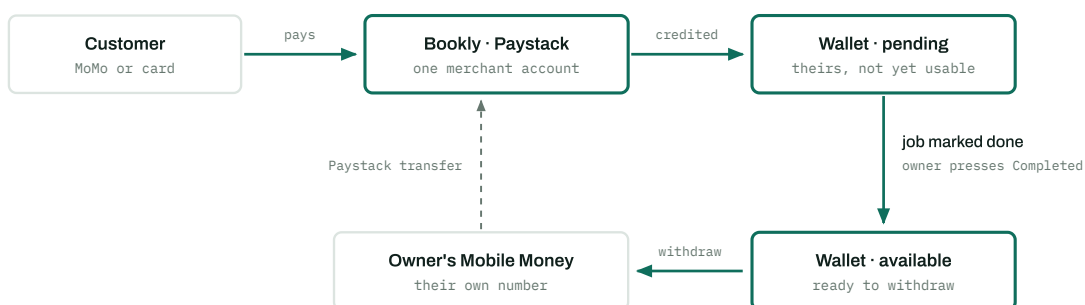
The users are salon owners, barbers and nail technicians running a business from a phone. Asking them to open a Paystack account, find a secret key and paste it into a settings page is where they stop. So Bookly does it for them and settles up afterwards.

Why a wallet rather than Paystack's split payments. Splits would settle each salon's share directly and Bookly would hold nothing — simpler and lower risk. But splits are immutable and refunds come out of the platform balance with no clawback from the subaccount. Bookly sells *deposits*, which are conditional money, and chargebacks exist whatever the refund policy says. A wallet with a clearing step is the only shape that survives a customer disputing a charge after the salon has been paid.

Tenants who already connected their own Paystack keep using it. Money they are already collecting must never silently start landing somewhere else, so the collection route is recorded per payment and both paths run side by side.

02

The flow



Money physically sits in one Paystack account throughout. The wallet records **whose it is**, which is a different question from where it sits.

The ledger

A balance is never a number anyone edits. It is the sum of an append-only log, and the cached columns exist only so a dashboard renders quickly — no withdrawal is ever authorised against them.

Accounts answer **one** question: who owns this money right now. They deliberately do not also track where it physically sits. Tracking both needs two-dimensional accounting, and the half-finished version of that is where sign errors hide.

ACCOUNT	MEANING
EXTERNAL	Outside Bookly — the customer's card, the owner's MoMo wallet. Its existence is what lets an inbound payment sum to zero.
TENANT_PENDING	Theirs, but the job is not done. Not withdrawable.
TENANT_AVAILABLE	Theirs, withdrawable now.
PAYOUT_PENDING	Committed to a transfer that has not confirmed.
PLATFORM_FEE	Bookly's revenue. A percentage in basis points, currently set to zero.

Every movement writes two or more rows that sum to zero, in one transaction. Money is moved, never created.

EVENT	MOVEMENT
Customer pays GHS 50	EXTERNAL -5000 · TENANT_PENDING +4900 · PLATFORM_FEE +100
Job marked done	TENANT_PENDING -4900 · TENANT_AVAILABLE +4900
Withdrawal requested	TENANT_AVAILABLE -4900 · PAYOUT_PENDING +4900
Transfer confirmed	PAYOUT_PENDING -4900 · EXTERNAL +4900
Transfer failed	PAYOUT_PENDING -4900 · TENANT_AVAILABLE +4900
Salon cancels	TENANT_PENDING -4900 · PLATFORM_FEE -100 · EXTERNAL +5000

Amounts are integer minor units — pesewas — throughout. A float loses money to rounding and the loss is unrecoverable, because nobody can say when it happened.

What happens to a deposit

Deposits are **non-refundable**. A customer who cannot make it reschedules, which keeps the same booking so their money simply travels with it — no ledger movement at all.

OUTCOME	WHERE THE MONEY GOES
Job completed	Released to the salon.
Customer no-show	Released to the salon. This is the entire point of taking a deposit.
Customer cancels	Forfeit to the salon. That is what makes holding the slot worth anything.
Customer reschedules	Travels with the booking. Nothing moves.
Salon cancels	Refunded to the customer, automatically.

WHY THE LAST ROW IS NOT A POLICY CHOICE

Keeping a customer's money for work nobody is going to do is indefensible however the terms are written — and the alternative to refunding is not "we keep it", it is a chargeback that costs the deposit plus a dispute fee plus standing with Paystack. Money held with no owner and no exit is a leak, so the refund is automatic rather than a balance waiting for someone to notice.

What stops it going wrong

RISK	CONTROL
Two withdrawals at once	Balance re-derived from the ledger <i>inside</i> a serializable transaction. Postgres aborts one of a colliding pair. Only one payout may be in flight at a time.
Overdrawing a balance	After every movement, balances are re-derived in the same transaction and a negative pot throws, rolling the write back. Conserving money per movement is not enough — two movements can each balance and still overdraw between them.
Refund and release colliding	Both take an atomic claim on the deposit before any provider call. One wins, the other stops.
Replayed webhooks	Every movement carries an idempotency key derived from the provider's reference, enforced by a unique index rather than an <code>if</code> that can lose a race.
Crediting money we never received	The collection route is recorded server-side when the link is created, and is the only thing allowed to decide whether a wallet may be credited.

Underpayment	A payment short of what was asked does not confirm the booking and does not credit.
Paying twice on a timeout	Only a definitive rejection returns the funds. An uncertain outcome leaves the payout in flight for the webhook to settle.
Account takeover	Changing the payout destination starts a 24-hour hold, is audited, and notifies the owner. Withdrawals are owner-only.
Marking work done that never happened	Booking transitions are a state machine, and a job cannot be completed before its start time.

FOUND BY REVIEW, NOT BY TESTS

An adversarial review of this path found a **critical** flaw: the collection route had been inferable from the transaction reference and the provider's metadata, both of which a tenant controls once they connect their own Paystack key. A tenant could have minted a payment in their own account and had Bookly credit and pay out real money. Every individual function behaved exactly as written, so no unit test would have caught it. It is fixed, and the inference path was deleted rather than patched.

06

Known limits

ITEM	POSITION
No hold before withdrawal	Funds clear the moment a job is marked done, by product decision. The state machine blocks instant fraud but not a patient version. A hold applied only to new tenants would close it without penalising established salons.
Open MEDIUM findings	Six, including staff being able to trigger a refund, and currency not being validated. Tracked in the build checklist.
Platform fee	Built as a percentage in basis points, set to zero until a rate is chosen.
Nothing is live	No platform Paystack account is configured, so none of this moves real money yet.

07

Before real money

Two of these are not engineering, and neither can be worked around in code.

PREREQUISITE	WHY
Written aggregator or marketplace approval from Paystack	Collecting on behalf of third parties without it risks the account being frozen — with tenant money inside it.
Registered business status	Paystack Transfers, which every payout depends on, are gated on it.
Close the open findings	Six MEDIUM and six LOW remain from the review.

Decide the fee

The mechanism is built; the number is a business decision.

Technical brief, September 2026. Describes the implementation as built, including the parts that are deliberately unfinished.

Holding funds on behalf of other businesses is regulated activity in Ghana under the Payment Systems and Services Act 2019. This document is not legal advice.